

Kuroyume Backend & Database Implementation Guide

A hands-on Windows 11 / PowerShell guide Updated 27 September 2026 - manual build edition

Read this first

Work through one small result at a time. This guide is for **you to build it yourself**. Each coding step names the exact file, gives code to paste, a command to run, and a result to check. Finish that check before moving on. Text such as `<YOUR_PASSWORD>` is a placeholder you replace with your own value. Never put real passwords or provider keys into a tracked file.

Your project now has two sibling folders:

```
C:\Users\joshj\OneDrive\Desktop\Projects\kuroyume\  
kuro-frontend\  Next.js and the guest library  
kuro-backend\   FastAPI, database models, first migration, partial auth work
```

This matters: commands for `kuro-backend` run **inside that folder**, and commands for `kuro-frontend` run inside its sibling. The older guide said to create `backend/` inside the frontend; do not create that extra folder. Your backend now has a `venv` environment, database models, a first migration, and partial auth files. If you are continuing the current project, skip the completed setup steps and begin at Phase 6. Earlier phases remain as setup reference; do not paste their starter examples over working files. The backend has not yet been made into a Git repository. Before deploying it, decide whether it will have its own Git repo or join a parent repo; the frontend's `.gitignore` does not protect files in its sibling folder.

The path you are working toward is:

```
Browser -> Next.js frontend -> FastAPI backend -> PostgreSQL  
|  
+--> external catalog provider when allowed
```

FastAPI handles accounts and account libraries. PostgreSQL saves them. Next.js keeps the UI and the existing browser-only guest library. Later, Alembic tracks changes to database tables. Provider catalog storage and bulk synchronization are **paused until written provider permission is clear**; you can complete the account work without that decision.

How to use a code block

When a step says **Paste into a file**, open that exact file in an editor, replace its contents with the block, and save it. When it says **add**, place the block at the named spot without deleting the rest of the file. A Python `__init__.py` file is just a marker that makes a folder importable; it can be empty. Copy long code blocks from the **Markdown guide**; PDF line wraps can change code. Code blocks are file contents, **not PowerShell commands**, unless labelled `powershell`.

A simple map of today's frontend

Need to find	Open in kuro-frontend
Provider HTTP calls and the small memory cache	lib/catalog/client.ts
Rankings, detail, search, seasons, recommendations	lib/catalog/service.ts
UI title data shapes	lib/catalog/types.ts
Browser-only guest library	lib/library/model.ts , lib/library/store.ts
Library controls and import/export	components/library/controls.tsx , components/library/views.tsx
Existing frontend checks	tests/ , .github/workflows/tests.yml

Phase 0 - Confirm your starting point

Step 0.1 - See what already exists

Make: Nothing yet. Open PowerShell and go to the frontend folder.

Run: `Set-Location` changes folders. `git status` reads the working tree; it does not change files. `rg` shows where the current data functions are called.

```
Set-Location 'C:\Users\joshj\OneDrive\Desktop\Projects\kuroyume\kuro-frontend'
git status --short
rg 'getDetail|saveTitle|useLibrary' app components lib
```

Check: You can point to the provider flow (`detail-page.tsx` -> `service.ts` -> `client.ts`) and the guest-save flow (`controls.tsx` -> `store.ts` -> browser `localStorage`). Write down anything `git status` shows so you do not overwrite your own work.

If it fails: If PowerShell says the folder does not exist, find where you moved `kuro-frontend` and substitute that path consistently. Do not create a new empty frontend folder.

☐ Step 0.1 complete

Phase 1 - Check the tools on Windows

Step 1.1 - Python, PostgreSQL, and pnpm

Make: Install only what the checks show is missing. Use the [Python Windows download](#) for Python 3.12 and the [PostgreSQL Windows installer page](#) for PostgreSQL. During Python setup, enable the launcher (`py`). During PostgreSQL setup, include command-line tools, record the port (usually 5432), and choose your own administrator password. Do not use or delete `kuro-frontend/.postgres` or `.backups`; they are unrelated existing local data.

Run: These commands inspect versions and the PostgreSQL Windows service.

```
py -3.12 --version
pnpm --version
Get-Command psql -ErrorAction SilentlyContinue
Get-Service postgresql* -ErrorAction SilentlyContinue
```

Check: Python prints 3.12.x, pnpm prints 11.5.2, and a PostgreSQL service reports `Running`. A missing `psql` command can mean it is simply not on `PATH`. Look for `C:\Program Files\PostgreSQL\<version>\bin\psql.exe`; use that full path in later commands if needed.

If it fails: If `python` opens the Microsoft Store, use `py -3.12` as shown. If PostgreSQL is installed but stopped, find the exact service name with `Get-Service postgresql*`; then start **that service** in an administrator PowerShell window. If port 5432 is already in use, record the port chosen during installation and use it consistently below.

☐ [Step 1.1 complete](#)

Phase 2 - Create two local databases

Step 2.1 - Add development and test databases

Know first: PostgreSQL is a running server. A *role* is its login; a *database* is a separate container of tables. You will create one pair for development and another for tests. These SQL statements change **your local PostgreSQL server**. Confirm the host before you run them. Choose your own strong passwords and do not copy the placeholders literally.

Run: This opens the PostgreSQL terminal as its administrator role. It prompts for the administrator password chosen during installation.

```
psql -U postgres -h 127.0.0.1 -p 5432 -d postgres
```

At the `postgres=#` prompt, run `\conninfo`. It must show your intended local host and port. Then run:

```
CREATE ROLE kuroyume_dev LOGIN PASSWORD '<YOUR_DEV_PASSWORD>';
CREATE ROLE kuroyume_test LOGIN PASSWORD '<YOUR_TEST_PASSWORD>';
CREATE DATABASE kuroyume_dev OWNER kuroyume_dev;
CREATE DATABASE kuroyume_test OWNER kuroyume_test;
\l kuroyume_*
\q
```

Check: From PowerShell, connect once with each new role. Each command prompts for its matching password. At each `psql` prompt, run `SELECT current_database(), current_user;` and then `\q`. The names must match the role and database you intended.

```
psql -U kuroyume_dev -h 127.0.0.1 -p 5432 -d kuroyume_dev
psql -U kuroyume_test -h 127.0.0.1 -p 5432 -d kuroyume_test
```

If it fails: `connection refused` means no server answered at that host/port. `password authentication failed` means the server answered but rejected the login. `database already exists` means you should inspect its owner and contents before deciding whether to use it; do not delete it just to repeat this step. If the password contains a single quote, use PostgreSQL's `\password <role>` after creating the role instead of embedding it in SQL.

☐ [Step 2.1 complete](#)

Step 2.2 - Practice reading the database

Make: Nothing. Connect to `kuroyume_dev` again with the preceding command.

Run inside psql: `\conninfo` shows which server/database you are in. `\dn` lists schemas (groups of tables). `\dt` lists tables. `SELECT` reads data. Kuroyume will later create a *private* schema for account data. A transaction means a group of changes is committed together or rolled back together.

```
\conninfo
\dn
SELECT current_database(), current_user;
\q
```

Check: You see `kuroyume_dev` and `kuroyume_test`, and no Kuroyume tables yet. This is your independent way to check later database steps.

☐ [Step 2.2 complete](#)

Phase 3 - Make the empty backend files work

Step 3.1 - Build and run the first FastAPI endpoint

Outcome: By the end of this step, `http://127.0.0.1:8000/health/live` returns `{ "status": "ok" }`. You do not need PostgreSQL code yet.

1. Go to the existing backend. You already have some empty files. `Set-Location` changes the current folder; `New-Item` creates missing folders and the three empty `__init__.py` marker files. It does not ask you to create a new `backend/` directory.

```
Set-Location 'C:\Users\joshj\OneDrive\Desktop\Projects\kuroyume\kuro-backend'
New-Item -ItemType Directory -Force -Path app\api, app\core | Out-Null
foreach ($file in @('app\__init__.py', 'app\api\__init__.py', 'app\core\__init__.py')) {
    if (-not (Test-Path -LiteralPath $file)) { New-Item -ItemType File -Path $file | Out-Null }
}
```

2. Fill `pyproject.toml`. Open `kuro-backend/pyproject.toml` in your editor (or run `notepad .\pyproject.toml`). Replace the empty file with this exact content and save. The `[project]` block is Python's package description. `dependencies` lists what `pip` must install. The `[build-system]` and `setuptools` sections make `pip install -e .` work from this folder. The version ranges keep the major versions compatible; pin the exact tested environment before deployment.

```
[build-system]
requires = ["setuptools>=75"]
build-backend = "setuptools.build_meta"

[project]
name = "kuroyume-backend"
version = "0.1.0"
requires-python = ">=3.12,<3.14"
dependencies = [
    "fastapi>=0.115,<1",
    "uvicorn[standard]>=0.30,<1",
    "sqlalchemy>=2.0,<2.1",
    "psycpg[binary]>=3.2,<4",
    "alembic>=1.13,<2",
    "pydantic-settings>=2.5,<3",
    "pwdlib[argon2]>=0.2,<1",
    "httpx>=0.27,<1",
    "email-validator>=2,<3",
]

[project.optional-dependencies]
test = ["pytest>=8,<10"]

[tool.setuptools.packages.find]
include = ["app*"]
```

3. Fill `app/main.py`. Open `kuro-backend/app/main.py` (or run `notepad .\app\main.py`), paste the following, and save. `FastAPI()` creates the app. `@app.get` maps a GET request to the function below it. This endpoint checks only that the Python web process is alive.

```
from fastapi import FastAPI

app = FastAPI(title="Kuroyume API")

@app.get("/health/live")
def live() -> dict[str, str]:
    return {"status": "ok"}
```

Leave `app/core/config.py` and `.env.example` empty for now; Step 3.2 fills them. Leave the `__init__.py` files empty. Your folder should now look like this:

```
kuro-backend/
  pyproject.toml
  .env.example
```

```

app/
  __init__.py
  main.py
  api/__init__.py
  core/__init__.py
  core/config.py

```

4. Create an isolated Python environment and install the package. The first command creates `.venv` inside `kuro-backend`; the second installs the packages named in `pyproject.toml` and the test extra. These commands can take several minutes and need an internet connection. They do not install anything into your frontend.

```

py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -e '[test]'

```

5. Start the API. Uvicorn is the small server program that listens on port 8000 and runs the FastAPI `app` variable from `app/main.py`. `--reload` restarts it after local code edits. Keep this PowerShell window open while testing.

```

.\.venv\Scripts\python.exe -m uvicorn app.main:app --reload --port 8000

```

6. Check in a second PowerShell window. The command should print `status : ok`. Opening `/docs` in your browser should show the interactive FastAPI page. Stop Uvicorn in the first window with `Ctrl+C` only after both checks work.

```

Invoke-RestMethod http://127.0.0.1:8000/health/live

```

If it fails: `py -3.12 missing` means Python 3.12 is not installed or the launcher was omitted. A `pip` build error often means `pyproject.toml` was not saved exactly. `Could not import module app.main` usually means the `app` folder, `__init__.py`, or `main.py` is missing, or you started the command outside `kuro-backend`. If port 8000 is busy, use `--port 8001` and test that port. You do not have to activate `.venv`; using its full `python.exe` path is enough.

☐ Step 3.1 complete - I saw `status : ok` and opened `/docs`

Step 3.2 - Fill the configuration files

Outcome: The backend can read local settings without storing real passwords in code or Git.

1. Fill `app/core/config.py`. Open the empty file and paste this. Pydantic Settings reads environment variables and then the local `.env` file. The path uses `config.py`'s location, so it works whether PowerShell starts from `kuro-backend` or another folder. The `@lru_cache` means the settings object is created once per process.

```

from functools import lru_cache
from pathlib import Path
from pydantic_settings import BaseSettings, SettingsConfigDict

BACKEND_DIR = Path(__file__).resolve().parents[2]

class Settings(BaseSettings):
    database_url: str = ""
    test_database_url: str = ""
    frontend_origin: str = "http://localhost:3000"
    cookie_secure: bool = False
    rapidapi_key: str = ""
    catalog_persistence_enabled: bool = False
    model_config = SettingsConfigDict(env_file=BACKEND_DIR / ".env", extra="ignore")

    @lru_cache
    def get_settings() -> Settings:
        return Settings()

```

2. Fill `.env.example`. This file is safe to share because it contains placeholders, not real secrets. It also reminds you which settings you need later. If your PostgreSQL port differs from 5432, change it in your local `.env` after copying.

```

DATABASE_URL=postgresql+psycopg://kuroyume_dev:<URL_ENCODED_DEV_PASSWORD>@127.0.0.1:5432/kuroyu ↵

```

```
me_dev
TEST_DATABASE_URL=postgresql+psycopg://kuroyume_test:<URL_ENCODED_TEST_PASSWORD>@127.0.0.1:5432 ↵
/kuroyume_test
FRONTEND_ORIGIN=http://localhost:3000
COOKIE_SECURE=false
CATALOG_PERSISTENCE_ENABLED=false
RAPIDAPI_KEY=
```

3. Protect the real .env file. Add these lines to `kuro-backend/.gitignore` (create the file if it does not exist). The frontend's `.gitignore` does not cover this sibling backend folder.

```
.venv/
.env
__pycache__/
*.pyc
.pytest_cache/
```

4. Make the local copy and fill passwords. `Copy-Item` makes a separate `.env`; `notepad` lets you edit it. Replace both password placeholders with the passwords you chose in Phase 2. If a password contains `@`, `:`, `/`, `%`, or other URL punctuation, URL-encode it first. The `urllib.parse.quote` command below prompts for a password and prints its encoded form; do not share its output.

```
Copy-Item .\.env.example .\.env
notepad .\.env
.\.venv\Scripts\python.exe -c "from urllib.parse import quote; import getpass; print(quote(getp
ass.getpass('Password to encode: '), safe=''))"
```

5. Check without printing secrets. This imports the settings, parses the development URL, and prints only host and database name. Expect `127.0.0.1 kuroyume_dev`. Do not print the entire URL. If you later put `kuro-backend` under Git, `git status --short` must never list `.env`.

```
.\.venv\Scripts\python.exe -c "from app.core.config import get_settings; from sqlalchemy.engine ↵
import make_url; u=make_url(get_settings().database_url); print(u.host, u.database)"
```

If it fails: `ValidationError` often means a malformed boolean or URL setting. A blank database name means `.env` was not saved or the placeholder was left in place. The config file is deliberately simple; it does not connect to PostgreSQL until Phase 4.

☐ [Step 3.2 complete - the settings check names my development database without showing a password](#)

Phase 4 - Prove that FastAPI can reach PostgreSQL

Step 4.1 - Add SQLAlchemy and a database health check

Outcome: `/health/ready` reports `ready` only when a real database query succeeds.

Know first: A SQLAlchemy `*engine*` holds connection settings and a small pool of reusable connections. A `*Session*` is one short unit of work for a request. Do not share one Session between all visitors. `SELECT 1` is a tiny read that proves the connection works without creating a table. The existing `/health/live` endpoint keeps checking only the web process.

1. Create the files. In `kuro-backend` PowerShell, create `app/db` and its marker file. Open the other two files in your editor and paste the blocks below.

```
New-Item -ItemType Directory -Force -Path app\db | Out-Null
if (-not (Test-Path -LiteralPath 'app\db\__init__.py')) { New-Item -ItemType File -Path 'app\db ↵
  \__init__.py' | Out-Null }
notepad .\app\db\base.py
notepad .\app\db\session.py
```

`app/db/base.py` gives all future table models one common parent:

```
from sqlalchemy.orm import DeclarativeBase

class Base(DeclarativeBase):
```

pass

`app/db/session.py` creates the engine once and provides one Session to each FastAPI request. The `with` block closes that Session after the request. A write endpoint later calls `commit()` deliberately; an exception triggers `rollback`.

```
from collections.abc import Iterator
from sqlalchemy import create_engine
from sqlalchemy.orm import Session, sessionmaker
from app.core.config import get_settings

engine = create_engine(get_settings().database_url, pool_pre_ping=True)
SessionLocal = sessionmaker(bind=engine, expire_on_commit=False)

def get_db() -> Iterator[Session]:
    with SessionLocal() as session:
        try:
            yield session
        except Exception:
            session.rollback()
            raise
```

2. Replace `app/main.py` with this version. It keeps `/health/live` and adds `/health/ready`. `Depends(get_db)` asks FastAPI to supply a Session. A Pydantic schema is not needed for this tiny health response; later API routes will use Pydantic to validate JSON.

```
from typing import Annotated
from fastapi import Depends, FastAPI
from sqlalchemy import text
from sqlalchemy.orm import Session
from app.db.session import get_db

app = FastAPI(title="Kuroyume API")

@app.get("/health/live")
def live() -> dict[str, str]:
    return {"status": "ok"}

@app.get("/health/ready")
def ready(db: Annotated[Session, Depends(get_db)]) -> dict[str, str]:
    db.execute(text("SELECT 1"))
    return {"status": "ready"}
```

3. Run and check. Start Uvicorn again from `kuro-backend`. In another window, run both requests. They should say `ok` and `ready`. This phase does not create any database tables.

```
.\.venv\Scripts\python.exe -m uvicorn app.main:app --reload --port 8000
```

```
Invoke-RestMethod http://127.0.0.1:8000/health/live
Invoke-RestMethod http://127.0.0.1:8000/health/ready
```

If it fails: `connection refused` means PostgreSQL is not listening at the configured host/port. `password authentication failed` means you reached PostgreSQL but it rejected the role/password. `database ... does not exist` means the URL names a different database from Phase 2. If `/health/live` fails too, look at the Uvicorn window for an import or startup error.

☐ Step 4.1 complete - both health checks respond, including the PostgreSQL check

Phase 5 - Make the first database tables

Step 5.1 - Describe account data in Python

Outcome: Five account tables are described in code. They do not exist in PostgreSQL until Step 5.2.

1. Make the files. Stay in `kuro-backend`. This command creates a model folder and its package marker. It leaves existing files alone.

```
New-Item -ItemType Directory -Force -Path app\models | Out-Null
if (-not (Test-Path app\models\__init__.py)) { New-Item -ItemType File app\models\__init__.py | Out-Null }
notepad .\app\models\private.py
```

Notepad may ask whether to create `private.py`; choose **Yes**. A `*model*` is a Python description of one database table. Paste this starter code into `private.py` and save it. It gives the five tables their essential fields and database rules. You can add more fields later through another reviewed migration.

```
from datetime import datetime
from uuid import UUID, uuid4
from sqlalchemy import CheckConstraint, DateTime, ForeignKey, Index, String, UniqueConstraint, func
from sqlalchemy.orm import Mapped, mapped_column
from app.db.base import Base

class User(Base):
    __tablename__ = "users"
    id: Mapped[UUID] = mapped_column(primary_key=True, default=uuid4)
    email: Mapped[str] = mapped_column(String(320), nullable=False)
    password_hash: Mapped[str] = mapped_column(String(255), nullable=False)
    verified_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), server_default=func.now())
    __table_args__ = (Index("uq_users_email_lower", func.lower(email), unique=True), {"schema": "private"})

class LoginSession(Base):
    __tablename__ = "sessions"
    id: Mapped[UUID] = mapped_column(primary_key=True, default=uuid4)
    user_id: Mapped[UUID] = mapped_column(ForeignKey("private.users.id"), nullable=False)
    token_hash: Mapped[str] = mapped_column(String(64), unique=True, nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), server_default=func.now())
    expires_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), nullable=False)
    revoked_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))
    __table_args__ = {"schema": "private"}

class EmailToken(Base):
    __tablename__ = "email_tokens"
    id: Mapped[UUID] = mapped_column(primary_key=True, default=uuid4)
    user_id: Mapped[UUID] = mapped_column(ForeignKey("private.users.id"), nullable=False)
    purpose: Mapped[str] = mapped_column(String(20), nullable=False)
    token_hash: Mapped[str] = mapped_column(String(64), unique=True, nullable=False)
    expires_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), nullable=False)
    used_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))
    __table_args__ = {"schema": "private"}

class Collection(Base):
    __tablename__ = "collections"
    id: Mapped[UUID] = mapped_column(primary_key=True, default=uuid4)
    user_id: Mapped[UUID] = mapped_column(ForeignKey("private.users.id"), nullable=False)
    name: Mapped[str] = mapped_column(String(100), nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), server_default=func.now())
    __table_args__ = (UniqueConstraint("user_id", "name"), {"schema": "private"})

class LibraryEntry(Base):
    __tablename__ = "library_entries"
    id: Mapped[UUID] = mapped_column(primary_key=True, default=uuid4)
    user_id: Mapped[UUID] = mapped_column(ForeignKey("private.users.id"), nullable=False)
    medium: Mapped[str] = mapped_column(String(10), nullable=False)
    provider_id: Mapped[str] = mapped_column(String(100), nullable=False)
    title_snapshot: Mapped[str] = mapped_column(String(500), nullable=False)
    status: Mapped[str] = mapped_column(String(30), nullable=False)
    progress: Mapped[int] = mapped_column(default=0)
    rating: Mapped[float | None]
```



```

favorite: Mapped[bool] = mapped_column(default=False)
notes: Mapped[str] = mapped_column(String(4000), default="")
version: Mapped[int] = mapped_column(default=1)
__table_args__ = (
    UniqueConstraint("user_id", "medium", "provider_id"),
    CheckConstraint("progress >= 0"),
    CheckConstraint("rating IS NULL OR (rating >= 0 AND rating <= 10)"),
    {"schema": "private"},
)

```

This is the table foundation, not the finished account feature. Later steps add request validation and ownership checks. Keep the allowed status names and the 2,000-entry limit aligned with the existing guest code in `kuro-frontend/lib/library/model.ts`.

Model/table in PostgreSQL <code>private</code> schema	Fields to add	Rule to check
<code>users</code>	UUID <code>id</code> , email, password hash, verified time, created time	Email is unique even when its letter case differs. Never save a plaintext password.
<code>sessions</code>	UUID <code>id</code> , <code>user_id</code> , token hash, created/expiry/revoked times	Each session points to an existing user.
<code>email_tokens</code>	UUID <code>id</code> , <code>user_id</code> , purpose, token hash, expiry/used times	Verification and reset links work once.
<code>collections</code>	UUID <code>id</code> , <code>user_id</code> , name, created time	A user's collection names are unique.
<code>library_entries</code>	UUID <code>id</code> , <code>user_id</code> , medium, provider ID, status, progress, rating, favorite, notes, title snapshot, version	One row per user + medium + provider ID; version begins at 1.

Use `sqlalchemy.orm.Mapped` and `mapped_column` for fields, and `ForeignKey('private.users.id')` for `user_id`. Set `__table_args__ = {'schema': 'private'}` on each model. Use the existing guest rules in `kuro-frontend/lib/library/model.ts` to choose valid statuses, rating range, note length and the 2,000-entry limit. Do not guess new status names. Keep provider content separate: a user's saved title must survive if provider data later expires.

2. Connect the models to Alembic. At the bottom of `app/models/__init__.py`, add imports for each model class you create. This is what lets the migration tool discover tables:

```
from app.models.private import User, LoginSession, EmailToken, Collection, LibraryEntry
```

Check: In PowerShell, run this import check from `kuro-backend`. It should print `models import` and no traceback. If you built only `User` so far, import only `User` until the other classes exist.

```
.\.env\Scripts\python.exe -c "import app.models; print('models import')"
```

If it fails: Read the last line of the traceback. `NameError` usually means a missing import in `private.py`; `ImportError` usually means the class name in `__init__.py` does not match. Fix that first. A model import does not create tables.

☐ Step 5.1 complete - all five model classes import

Step 5.2 - Turn the models into database tables

Know first: A *migration* is a saved recipe for changing tables. Alembic can draft that recipe from models, but you must read it before running it. Use your **development** database for these commands.

1. Create Alembic's starter files. Run this once from `kuro-backend`:

```
.\.env\Scripts\python.exe -m alembic init alembic
```

It creates `alembic.ini`, `alembic/env.py`, and `alembic/versions/`. Do not run `init` again after those exist.

2. Tell Alembic where to look. Open `alembic/env.py`. Keep the generated offline/online functions, but add these imports near the top and replace its `target_metadata = None` line with the following:

```
from app.core.config import get_settings
from app.db.base import Base
import app.models # registers all table models

target_metadata = Base.metadata
config.set_main_option("sqlalchemy.url", get_settings().database_url.replace("%", "%%"))
```

Inside the generated `context.configure(...)` calls, add `include_schemas=True`. This tells Alembic about the `private` schema. Confirm `app/core/config.py` actually exports `get_settings()` and its `database_url` field as Step 3.2 shows. `alembic.ini` may still contain a sample URL; the `env.py` line above replaces it at run time.

3. Draft and inspect. The first command writes a new Python file under `alembic/versions/`; the second lists it so you can open the exact filename.

```
.\venv\Scripts\python.exe -m alembic revision --autogenerate -m "initial private tables"
Get-ChildItem .\alembic\versions\*.py | Select-Object Name
```

Open that new file. In its `upgrade()` function, add `op.execute('CREATE SCHEMA IF NOT EXISTS private')` **before** the table creation calls. Confirm it creates only your five tables, keys and indexes. If it contains `drop_table` for something you did not create, stop and check the database URL and imported models. The initial `downgrade()` is only a development aid; it may delete data.

4. Apply and inspect. `upgrade head` changes the database named by `.env`. Before running it, check that `.env` names `kuroyume_dev`.

```
Select-String -Path .\env -Pattern '^DATABASE_URL=' | ForEach-Object { $_.Line -replace ':[^:@ ↵
/]+@', ':[***@' }
.\venv\Scripts\python.exe -m alembic upgrade head
.\venv\Scripts\python.exe -m alembic current
```

Connect with `psql` as in Phase 2 and run `\dt private.*`. You should see five tables. Then run `\q` to leave `psql`.

If it fails: An empty revision means Alembic did not import your models. `schema "private" does not exist` means the schema creation line is missing or below the table calls. `relation private.users does not exist` usually means you connected to a different database or did not run the migration.

☐ Step 5.2 complete - `alembic current` has a revision and `\dt private.*` lists five tables

Phase 6 - Build accounts yourself

This phase uses the backend you have **now**: `venv/`, working `/health` and `/health/db`, five account tables, and an empty `app/api/auth.py`. Work inside `kuro-backend`. The Python blocks below are **file contents**. Paste them into the named file in your editor and save; only blocks labelled `powershell` go into PowerShell. Complete each check before moving on.

```
Set-Location 'C:\Users\joshj\OneDrive\Desktop\Projects\kuroyume\kuro-backend'
Test-Path .\venv\Scripts\python.exe
```

The second command must say `True`. Keep the backend running in one PowerShell window when you test endpoints:

```
.\venv\Scripts\python.exe -m uvicorn app.main:app --reload --port 8000
```

Use a second PowerShell window for the checks. In that window, run the same `Set-Location` command above before any file-based check. Stop the server with `Ctrl+C` when you finish.

Step 6.1 - Make signup, login, /me, and logout

1. Replace `app/schemas/auth.py`. Open it with notepad `.\app\schemas\auth.py`. The existing file has a `verified` field that does not exist directly on the `User` table; the route below calculates it from `verified_at`. Paste and save:

```
from uuid import UUID
from pydantic import BaseModel, EmailStr, Field

class RegisterRequest(BaseModel):
    email: EmailStr
    password: str = Field(min_length=12, max_length=128)

class LoginRequest(BaseModel):
    email: EmailStr
    password: str

class UserResponse(BaseModel):
    id: UUID
    email: EmailStr
    verified: bool
```

2. Keep `app/services/auth.py` **as it is**. It already has `hash_password`, `verify_password`, and `create_session_token`. You will use those functions in the route. Do not create a second password helper.

3. Create `app/api/deps.py`. Run notepad `.\app\api\deps.py`, choose **Yes** if Notepad asks to create it, and paste:

```
from datetime import datetime, timezone
from hashlib import sha256
from typing import Annotated

from fastapi import Depends, HTTPException, Request
from sqlalchemy import select
from sqlalchemy.orm import Session

from app.db.session import get_db
from app.models.private import LoginSession, User

def get_login(
    request: Request,
    db: Annotated[Session, Depends(get_db)],
) -> tuple[User, LoginSession, str]:
    token = request.cookies.get("kuroyume_session")
    if not token:
        raise HTTPException(status_code=401, detail="Sign in required")
    digest = sha256(token.encode("utf-8")).hexdigest()
    session = db.scalar(
        select(LoginSession).where(LoginSession.token_hash == digest)
    )
    if session is None:
        raise HTTPException(status_code=401, detail="Sign in required")
    expires_at = session.expires_at
    if expires_at.tzinfo is None:
        expires_at = expires_at.replace(tzinfo=timezone.utc)
    if session.revoked_at is not None or expires_at <= datetime.now(timezone.utc):
        raise HTTPException(status_code=401, detail="Sign in required")
    user = db.get(User, session.user_id)
    if user is None:
        raise HTTPException(status_code=401, detail="Sign in required")
    return user, session, token

def current_user(
    login: Annotated[tuple[User, LoginSession, str], Depends(get_login)],
) -> User:
    return login[0]
```

`get_login` reads the cookie and finds its matching unexpired database session. `current_user` is what later private routes use. A failed lookup returns HTTP 401.

4. Fill the empty app/api/auth.py. Run notepad .\app\api\auth.py, paste and save:

```
from datetime import datetime, timedelta, timezone
from typing import Annotated

from fastapi import APIRouter, Depends, HTTPException, Response
from sqlalchemy import func, select
from sqlalchemy.exc import IntegrityError
from sqlalchemy.orm import Session

from app.api.deps import current_user, get_login
from app.core.config import get_settings
from app.db.session import get_db
from app.models.private import LoginSession, User
from app.schemas.auth import LoginRequest, RegisterRequest, UserResponse
from app.services.auth import create_session_token, hash_password, verify_password

router = APIRouter(prefix="/v1/auth", tags=["auth"])
me_router = APIRouter(tags=["account"])
COOKIE_NAME = "kuroyume_session"
SESSION_SECONDS = 7 * 24 * 60 * 60

def public_user(user: User) -> UserResponse:
    return UserResponse(
        id=user.id, email=user.email, verified=user.verified_at is not None
    )

@router.post("/signup", response_model=UserResponse, status_code=201)
def signup(
    body: RegisterRequest,
    db: Annotated[Session, Depends(get_db)],
) -> UserResponse:
    user = User(
        email=str(body.email).strip().lower(),
        password_hash=hash_password(body.password),
    )
    db.add(user)
    try:
        db.commit()
    except IntegrityError:
        db.rollback()
        raise HTTPException(status_code=409, detail="Account already exists")
    db.refresh(user)
    return public_user(user)

@router.post("/login", response_model=UserResponse)
def login(
    body: LoginRequest,
    response: Response,
    db: Annotated[Session, Depends(get_db)],
) -> UserResponse:
    email = str(body.email).strip().lower()
    user = db.scalar(select(User).where(func.lower(User.email) == email))
    if user is None or not verify_password(body.password, user.password_hash):
        raise HTTPException(status_code=401, detail="Invalid email or password")
    token, digest = create_session_token()
    db.add(LoginSession(
        user_id=user.id,
        token_hash=digest,
        expires_at=datetime.now(timezone.utc) + timedelta(seconds=SESSION_SECONDS),
    ))
    db.commit()
    response.set_cookie(
        key=COOKIE_NAME,
        value=token,
        httponly=True,
        secure=get_settings().cookie_secure,
        samesite="lax",
        max_age=SESSION_SECONDS,
        path="/",
    )
)
```

```

    return public_user(user)

@me_router.get("/v1/me", response_model=UserResponse)
def me(user: Annotated[User, Depends(current_user)]) -> UserResponse:
    return public_user(user)

@router.post("/logout")
def logout(
    response: Response,
    db: Annotated[Session, Depends(get_db)],
    login: Annotated[tuple[User, LoginSession, str], Depends(get_login)],
) -> dict[str, str]:
    login[1].revoked_at = datetime.now(timezone.utc)
    db.commit()
    response.delete_cookie(COOKIE_NAME, path="/")
    return {"status": "signed out"}

```

5. Register the routes. Replace `app/main.py` with the whole block below. It keeps your current health addresses and fixes the app title spelling. A router is just a group of related URLs.

```

from typing import Annotated
from fastapi import Depends, FastAPI
from sqlalchemy import text
from sqlalchemy.orm import Session
from app.api.auth import me_router, router as auth_router
from app.db.session import get_db

app = FastAPI(title="Kuroyume API")
app.include_router(auth_router)
app.include_router(me_router)

@app.get("/health")
def health() -> dict[str, str]:
    return {"status": "ok"}

@app.get("/health/db")
def ready(db: Annotated[Session, Depends(get_db)]) -> dict[str, str]:
    db.execute(text("SELECT 1"))
    return {"status": "ready"}

```

6. Run and check. First check that Python can import the app; it should print `imports ok`. Then start Uvicorn in this window.

```

.\venv\Scripts\python.exe -c "from app.main import app; print('imports ok')"
.\venv\Scripts\python.exe -m uvicorn app.main:app --reload --port 8000

```

Open <http://localhost:8000/docs>. In a second PowerShell window, use these commands to keep one login cookie across requests. The signup email needs to be new each time you rerun the block.

```

$api = 'http://localhost:8000'
$browser = New-Object Microsoft.PowerShell.Commands.WebRequestSession
$body = @{ email = 'guide-test-1@example.com'; password = 'LocalOnlyPassphrase123!' } | Convert -
To-Json
Invoke-RestMethod "$api/v1/auth/signup" -Method Post -ContentType 'application/json' -Body $bod
y
Invoke-RestMethod "$api/v1/auth/login" -Method Post -ContentType 'application/json' -Body $body -
WebSession $browser
Invoke-RestMethod "$api/v1/me" -WebSession $browser
Invoke-RestMethod "$api/v1/auth/logout" -Method Post -WebSession $browser

```

The signup and login responses should show your test email. `/v1/me` should work before logout. After logout, repeat `/v1/me`; it should return 401. PowerShell showing an error for that last request is expected. If signup says 409, change `guide-test-1` to a new test number and run again. This example password is for a local throwaway account only.

□ Step 6.1 complete - signup, login, `/v1/me`, and logout behave as above

Step 6.2 - Add verification and password reset for local development

You need somewhere to see a test email. This step writes development-only links into `.local-mail/` inside the backend. The folder is ignored by Git and must never be used for real users. Production email delivery is a separate deployment decision in Phase 11.

1. Add the settings. Open `app/core/config.py`. Add this line inside `class Settings` beside the other settings:

```
environment: str = "development"
```

Open `.env.example` and add `ENVIRONMENT=development`. Open `.gitignore` and add `.local-mail/` on its own line. Your local `.env` may omit the new setting because the default is `development`. Do not change it to production until an actual mail sender exists.

2. Create `app/services/mail.py`. Run notepad `.\app\services\mail.py`, paste and save:

```
from pathlib import Path
from app.core.config import BACKEND_DIR, get_settings

def save_local_email(kind: str, address: str, link: str) -> Path:
    if get_settings().environment != "development":
        raise RuntimeError("Configure a real mail sender before production")
    folder = BACKEND_DIR / ".local-mail"
    folder.mkdir(exist_ok=True)
    target = folder / f"{kind}-latest.txt"
    target.write_text(f"To: {address}\n\nOpen this link: {link}\n", encoding="utf-8")
    return target
```

3. Add request shapes to the bottom of `app/schemas/auth.py`. Paste these after `UserResponse`:

```
class EmailRequest(BaseModel):
    email: EmailStr

class TokenRequest(BaseModel):
    token: str

class ResetRequest(BaseModel):
    token: str
    new_password: str = Field(min_length=12, max_length=128)
```

4. Create `app/api/recovery.py`. This file creates one-use, expiring tokens. For an unknown address, request routes still return the same message. Paste the whole block:

```
from datetime import datetime, timedelta, timezone
from hashlib import sha256
from secrets import token_urlsafe
from typing import Annotated

from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy import func, select, update
from sqlalchemy.orm import Session

from app.core.config import get_settings
from app.db.session import get_db
from app.models.private import EmailToken, LoginSession, User
from app.schemas.auth import EmailRequest, ResetRequest, TokenRequest
from app.services.auth import hash_password
from app.services.mail import save_local_email

router = APIRouter(prefix="/v1/auth", tags=["recovery"])
GENERIC = {"status": "If the address has an account, a link is on its way"}

def issue(db: Session, email: str, purpose: str) -> dict[str, str]:
    if get_settings().environment != "development":
        raise HTTPException(status_code=503, detail="Mail delivery is not configured")
    user = db.scalar(select(User).where(func.lower(User.email) == email.lower()))
    if user is None:
        return GENERIC
    raw = token_urlsafe(32)
    db.add(EmailToken(
```

```

        user_id=user.id,
        purpose=purpose,
        token_hash=sha256(raw.encode("utf-8")).hexdigest(),
        expires_at=datetime.now(timezone.utc) + timedelta(minutes=30),
    ))
    db.commit()
    link = f"{get_settings().frontend_origin}/account/{purpose}?token={raw}"
    save_local_email(purpose, user.email, link)
    return GENERIC

def consume(db: Session, raw: str, purpose: str) -> tuple[User, EmailToken]:
    digest = sha256(raw.encode("utf-8")).hexdigest()
    row = db.scalar(select(EmailToken).where(
        EmailToken.token_hash == digest,
        EmailToken.purpose == purpose,
    ).with_for_update())
    if row is None or row.used_at is not None:
        raise HTTPException(status_code=400, detail="Invalid or expired link")
    expires_at = row.expires_at
    if expires_at.tzinfo is None:
        expires_at = expires_at.replace(tzinfo=timezone.utc)
    if expires_at <= datetime.now(timezone.utc):
        raise HTTPException(status_code=400, detail="Invalid or expired link")
    user = db.get(User, row.user_id)
    if user is None:
        raise HTTPException(status_code=400, detail="Invalid or expired link")
    return user, row

@router.post("/verification/request")
def request_verification(
    body: EmailRequest, db: Annotated[Session, Depends(get_db)]
) -> dict[str, str]:
    return issue(db, str(body.email), "verification")

@router.post("/verification/confirm")
def confirm_verification(
    body: TokenRequest, db: Annotated[Session, Depends(get_db)]
) -> dict[str, str]:
    user, row = consume(db, body.token, "verification")
    user.verified_at = row.used_at = datetime.now(timezone.utc)
    db.commit()
    return {"status": "verified"}

@router.post("/password/forgot")
def forgot_password(
    body: EmailRequest, db: Annotated[Session, Depends(get_db)]
) -> dict[str, str]:
    return issue(db, str(body.email), "reset")

@router.post("/password/reset")
def reset_password(
    body: ResetRequest, db: Annotated[Session, Depends(get_db)]
) -> dict[str, str]:
    user, row = consume(db, body.token, "reset")
    now = datetime.now(timezone.utc)
    user.password_hash = hash_password(body.new_password)
    row.used_at = now
    db.execute(update(LoginSession).where(
        LoginSession.user_id == user.id,
        LoginSession.revoked_at.is_(None),
    ).values(revoked_at=now))
    db.commit()
    return {"status": "password changed"}

```

5. Register the new router. In `app/main.py`, add this import beside the existing auth import:

```
from app.api.recovery import router as recovery_router
```

Add `app.include_router(recovery_router)` beside the other `include_router` lines. Save, then run the import check again. Open `/docs`; the four new routes should appear.

6. Check locally. In a second PowerShell window, request a link for the test email from Step 6.1:

```
$api = 'http://localhost:8000'
$email = @{ email = 'guide-test-1@example.com' } | ConvertTo-Json
Invoke-RestMethod "$api/v1/auth/verification/request" -Method Post -ContentType 'application/json' -Body $email
Get-Content .\local-mail\verification-latest.txt
```

The file contains a URL with `token=...`. Copy only the token value after `token=`. Put it into `$token` below and run `confirm` twice. The first succeeds; the second must say `Invalid or expired link`.

```
$token = '<PASTE_TOKEN_VALUE_HERE>'
$confirm = @{ token = $token } | ConvertTo-Json
Invoke-RestMethod "$api/v1/auth/verification/confirm" -Method Post -ContentType 'application/json' -Body $confirm
```

Repeat with `/v1/auth/password/forgot`, read `.local-mail/reset-latest.txt`, and send its token plus a new password to `/v1/auth/password/reset` in `/docs`. Log in with the new password; the old one must fail. **The local email file is a development aid. Do not deploy it or use real addresses with it.**

□ Step 6.2 complete - each link works once, and reset invalidates the old password

Step 6.3 - Protect signed-in writes with a CSRF token

1. Add this block to the bottom of `app/api/deps.py`. It derives a second token from the random login cookie. The browser frontend can read this second token from a safe GET request; another site cannot read that response when CORS is limited to your frontend.

```
from secrets import compare_digest
from app.core.config import get_settings

def csrf_for(raw_session_token: str) -> str:
    return sha256(("csrf:" + raw_session_token).encode("utf-8")).hexdigest()

def require_write(
    request: Request,
    login: Annotated[tuple[User, LoginSession, str], Depends(get_login)],
) -> tuple[User, LoginSession, str]:
    if request.headers.get("origin") != get_settings().frontend_origin:
        raise HTTPException(status_code=403, detail="Wrong origin")
    supplied = request.headers.get("x-csrf-token", "")
    if not compare_digest(supplied, csrf_for(login[2])):
        raise HTTPException(status_code=403, detail="Missing or invalid CSRF token")
    return login
```

2. Make the token available. In `app/api/auth.py`, add `csrf_for` to the existing import from `app.api.deps`. Then add this route at the bottom:

```
@router.get("/csrf")
def csrf(
    login: Annotated[tuple[User, LoginSession, str], Depends(get_login)],
) -> dict[str, str]:
    return {"csrf_token": csrf_for(login[2])}
```

In the `logout` function, change only `Depends(get_login)` to `Depends(require_write)`, and add `require_write` to the same import line. This makes `logout` require both a valid session and the CSRF/Origin checks. Later library writes use `require_write` too.

3. Allow the exact frontend origin. In `app/main.py`, add the two imports below near the top:

```
from fastapi.middleware.cors import CORSMiddleware
from app.core.config import get_settings
```

Just after `app = FastAPI(...)`, add:

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=[get_settings().frontend_origin],
```



```

    allow_credentials=True,
    allow_methods=["GET", "POST", "PUT", "PATCH", "DELETE"],
    allow_headers=["Content-Type", "X-CSRF-Token"],
)

```

4. Test one protected write. Log in again with a new `WebRequestSession`, then read `/v1/auth/csrf`. Try logout without headers; it must return 403. Then supply the exact Origin and token:

```

$api = 'http://localhost:8000'
$browser = New-Object Microsoft.PowerShell.Commands.WebRequestSession
$loginBody = @{ email = 'guide-test-1@example.com'; password = 'LocalOnlyPassphrase123!' } | ConvertTo-Json
Invoke-RestMethod "$api/v1/auth/login" -Method Post -ContentType 'application/json' -Body $loginBody -WebSession $browser
$csrf = (Invoke-RestMethod "$api/v1/auth/csrf" -WebSession $browser).csrf_token
$headers = @{ Origin = 'http://localhost:3000'; 'X-CSRF-Token' = $csrf }
Invoke-RestMethod "$api/v1/auth/logout" -Method Post -WebSession $browser -Headers $headers

```

If you changed the test password in Step 6.2, use that new password in `$loginBody`. If `.env` uses a different `FRONTEND_ORIGIN`, put that exact value in `$headers.Origin`. The frontend helper in Phase 10 will fetch and send this token automatically.

☐ Step 6.3 complete - logout rejects a missing token and accepts a valid one

Phase 7 - Save an account library without losing guest mode

The current guest library stays in `kuro-frontend/lib/library/store.ts`. This phase adds **separate** server routes for signed-in users. Do not edit or clear `localStorage` yet.

Step 7.1 - Add save, read, delete, and collections

1. Link entries to collections. The `collections` table exists, but `library_entries` does not yet point to it. Open `app/models/private.py`. Inside class `LibraryEntry`, immediately after its `user_id` line, add:

```
collection_id: Mapped[UUID | None] = mapped_column(ForeignKey("private.collections.id"))
```

Save the file. Create a migration from `kuro-backend`:

```

.\venv\Scripts\python.exe -m alembic revision --autogenerate -m "link library collections"
Get-ChildItem .\alembic\versions\*.py | Sort-Object LastWriteTime -Descending | Select-Object -First 1 FullName

```

Open the new migration. Its `upgrade()` should **only** add nullable `collection_id` to `private.library_entries` and its foreign key. If it drops a table or changes unrelated data, stop and check the database URL/model import before proceeding. Once reviewed:

```

.\venv\Scripts\python.exe -m alembic upgrade head
.\venv\Scripts\python.exe -m alembic current

```

2. Create `app/schemas/library.py`. Run notepad `.\app\schemas\library.py`, paste and save. These are the fields a visitor can send or receive. The status names and limits match `kuro-frontend/lib/library/model.ts`.

```

from typing import Literal
from uuid import UUID
from pydantic import BaseModel, Field

Medium = Literal["anime", "manga"]
Status = Literal["planned", "watching", "reading", "completed", "on-hold", "dropped"]

class EntryInput(BaseModel):
    title: str = Field(min_length=1, max_length=500)
    status: Status = "planned"
    progress: int = Field(default=0, ge=0, le=100000)
    rating: int = Field(default=0, ge=0, le=10)
    favorite: bool = False
    notes: str = Field(default="", max_length=4000)

```

```

collection_id: UUID | None = None
version: int | None = Field(default=None, ge=1)

class EntryOutput(EntryInput):
    id: UUID
    medium: Medium
    provider_id: int
    version: int

class CollectionInput(BaseModel):
    name: str = Field(min_length=1, max_length=60)

class CollectionOutput(CollectionInput):
    id: UUID

```

3. Create `app/api/library.py`. Run notepad `.\app\api\library.py`, paste and save. Every query includes the signed-in user's ID. That is the rule that stops one account from reading another account's entries.

```

from typing import Annotated

from fastapi import APIRouter, Depends, HTTPException, Path, Query, Response
from sqlalchemy import func, select
from sqlalchemy.orm import Session

from app.api.deps import current_user, require_write
from app.db.session import get_db
from app.models.private import Collection, LibraryEntry, LoginSession, User
from app.schemas.library import EntryInput, EntryOutput, Medium

router = APIRouter(prefix="/v1/library", tags=["library"])
Login = tuple(User, LoginSession, str)

def view(row: LibraryEntry) -> EntryOutput:
    return EntryOutput(
        id=row.id, medium=row.medium, provider_id=int(row.provider_id),
        title=row.title_snapshot, status=row.status, progress=row.progress,
        rating=int(row.rating or 0), favorite=row.favorite,
        notes=row.notes, collection_id=row.collection_id, version=row.version,
    )

@router.get("", response_model=list[EntryOutput])
def list_entries(
    db: Annotated[Session, Depends(get_db)],
    user: Annotated[User, Depends(current_user)],
    limit: Annotated[int, Query(ge=1, le=100)] = 50,
    offset: Annotated[int, Query(ge=0)] = 0,
) -> list[EntryOutput]:
    rows = db.scalars(
        select(LibraryEntry)
        .where(LibraryEntry.user_id == user.id)
        .order_by(LibraryEntry.id)
        .limit(limit).offset(offset)
    ).all()
    return [view(row) for row in rows]

@router.put("/{medium}/{provider_id}", response_model=EntryOutput)
def save_entry(
    medium: Medium,
    provider_id: Annotated[int, Path(gt=0)],
    body: EntryInput,
    db: Annotated[Session, Depends(get_db)],
    login: Annotated[Login, Depends(require_write)],
) -> EntryOutput:
    user = login[0]
    if body.collection_id is not None:
        collection = db.get(Collection, body.collection_id)
        if collection is None or collection.user_id != user.id:
            raise HTTPException(status_code=404, detail="Collection not found")
    row = db.scalar(select(LibraryEntry).where(
        LibraryEntry.user_id == user.id,
        LibraryEntry.medium == medium,

```

```

        LibraryEntry.provider_id == str(provider_id),
    ).with_for_update()
    if row is None:
        count = db.scalar(select(func.count()).select_from(LibraryEntry).where(
            LibraryEntry.user_id == user.id
        ))
        if count >= 2000:
            raise HTTPException(status_code=400, detail="Library is full")
        row = LibraryEntry(
            user_id=user.id, medium=medium, provider_id=str(provider_id),
            title_snapshot=body.title, status=body.status,
            progress=body.progress, rating=body.rating,
            favorite=body.favorite, notes=body.notes,
            collection_id=body.collection_id, version=1,
        )
        db.add(row)
    else:
        if body.version != row.version:
            raise HTTPException(status_code=409, detail="Entry changed; reload it")
        row.title_snapshot = body.title
        row.status = body.status
        row.progress = body.progress
        row.rating = body.rating
        row.favorite = body.favorite
        row.notes = body.notes
        row.collection_id = body.collection_id
        row.version += 1
    db.commit()
    db.refresh(row)
    return view(row)

@router.delete("/{medium}/{provider_id}", status_code=204)
def delete_entry(
    medium: Medium,
    provider_id: Annotated[int, Path(gt=0)],
    db: Annotated[Session, Depends(get_db)],
    login: Annotated[Login, Depends(require_write)],
) -> Response:
    row = db.scalar(select(LibraryEntry).where(
        LibraryEntry.user_id == login[0].id,
        LibraryEntry.medium == medium,
        LibraryEntry.provider_id == str(provider_id),
    ))
    if row is None:
        raise HTTPException(status_code=404, detail="Entry not found")
    db.delete(row)
    db.commit()
    return Response(status_code=204)

```

4. Create `app/api/collections.py`. This creates and lists the signed-in user's collections. Save one collection first, then use its returned `id` as `collection_id` when saving an entry.

```

from typing import Annotated

from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy import select
from sqlalchemy.exc import IntegrityError
from sqlalchemy.orm import Session

from app.api.deps import current_user, require_write
from app.db.session import get_db
from app.models.private import Collection, LoginSession, User
from app.schemas.library import CollectionInput, CollectionOutput

router = APIRouter(prefix="/v1/collections", tags=["collections"])
Login = tuple[User, LoginSession, str]

def view(row: Collection) -> CollectionOutput:
    return CollectionOutput(id=row.id, name=row.name)

@router.get("", response_model=list[CollectionOutput])

```

```
def list_collections(
    db: Annotated[Session, Depends(get_db)],
    user: Annotated[User, Depends(current_user)],
) -> list[CollectionOutput]:
    rows = db.scalars(select(Collection).where(Collection.user_id == user.id)).all()
    return [view(row) for row in rows]

@router.post("", response_model=CollectionOutput, status_code=201)
def add_collection(
    body: CollectionInput,
    db: Annotated[Session, Depends(get_db)],
    login: Annotated[Login, Depends(require_write)],
) -> CollectionOutput:
    row = Collection(user_id=login[0].id, name=body.name.strip())
    db.add(row)
    try:
        db.commit()
    except IntegrityError:
        db.rollback()
        raise HTTPException(status_code=409, detail="Collection already exists")
    db.refresh(row)
    return view(row)
```

5. Register the routers. Add these imports near the top of `app/main.py`:

```
from app.api.library import router as library_router
from app.api.collections import router as collections_router
```

Add these after the existing `app.include_router(...)` lines:

```
app.include_router(library_router)
app.include_router(collections_router)
```

6. Check. Run the import check and start Uvicorn. `/docs` should show library and collection routes. Sign in, fetch `/v1/auth/csrf`, and send both Origin and CSRF on writes:

```
.\venv\Scripts\python.exe -c "from app.main import app; print('imports ok')"
```

```
$api = 'http://localhost:8000'
```

```
$browser = New-Object Microsoft.PowerShell.Commands.WebRequestSession
```

```
$loginBody = @{ email = 'guide-test-1@example.com'; password = 'LocalOnlyPassphrase123!' } | ConvertTo-Json
```

```
Invoke-RestMethod "$api/v1/auth/login" -Method Post -ContentType 'application/json' -Body $loginBody -WebSession $browser
```

```
$csrf = (Invoke-RestMethod "$api/v1/auth/csrf" -WebSession $browser).csrf_token
```

```
$headers = @{ Origin = 'http://localhost:3000'; 'X-CSRF-Token' = $csrf }
```

```
$entry = @{ title = 'Local test anime'; status = 'planned'; progress = 0; rating = 0; favorite = $false; notes = '' } | ConvertTo-Json
```

```
Invoke-RestMethod "$api/v1/library/anime/1" -Method Put -ContentType 'application/json' -Body $entry -Headers $headers -WebSession $browser
```

```
Invoke-RestMethod "$api/v1/library" -WebSession $browser
```

If you changed the test password in Step 6.2, replace `LocalOnlyPassphrase123!` above with that new password. The saved row should appear with `version: 1`. To edit it, send the same fields with `version: 1`; the response should show `version: 2`. Sending `version: 1` again must return 409. Sign in as a different local account and request `/v1/library`; it must not show the first account's row. A 403 write usually means the Origin or CSRF header does not match Step 6.3.

□ Step 7.1 complete - saving works, versions prevent stale edits, and accounts stay separate

Step 7.2 - Preview and import guest entries only after confirmation

The browser's guest export is a JSON file with `version`, `entries`, `recent`, and `compare`. You can export one from the existing guest library UI in the frontend. **Preview** checks it and writes nothing. **Confirm** sends the same file again and inserts only missing entries. The original guest file and `localStorage` stay untouched.

1. Create `app/api/imports.py`. Open the file with Notepad, paste this block, and save:

```
import json
```

```

from datetime import datetime, timezone
from typing import Annotated

from fastapi import APIRouter, Depends, HTTPException
from pydantic import ValidationError
from sqlalchemy import func, select
from sqlalchemy.orm import Session

from app.api.deps import current_user, require_write
from app.db.session import get_db
from app.models.private import Collection, LibraryEntry, LoginSession, User
from app.schemas.library import EntryInput

router = APIRouter(prefix="/v1/library-import", tags=["import and export"])
Login = tuple[User, LoginSession, str]

def read_guest(payload: dict) -> list[tuple[str, int, EntryInput, str]]:
    if len(json.dumps(payload).encode("utf-8")) > 2_000_000:
        raise HTTPException(status_code=400, detail="File is over 2 MB")
    entries = payload.get("entries")
    if payload.get("version") != 1 or not isinstance(entries, dict):
        raise HTTPException(status_code=400, detail="Unsupported guest file")
    if len(entries) > 2000:
        raise HTTPException(status_code=400, detail="Too many entries")
    output = []
    for raw in entries.values():
        if not isinstance(raw, dict) or not isinstance(raw.get("title"), dict):
            raise HTTPException(status_code=400, detail="Invalid guest entry")
        title = raw["title"]
        medium, provider_id = title.get("medium"), title.get("id")
        if medium not in ("anime", "manga") or type(provider_id) is not int or provider_id <= 0:
            raise HTTPException(status_code=400, detail="Invalid title ID")
        try:
            item = EntryInput(
                title=title.get("title"), status=raw.get("status"),
                progress=raw.get("progress"), rating=raw.get("rating"),
                favorite=raw.get("favorite"), notes=raw.get("notes"),
            )
        except ValidationError:
            raise HTTPException(status_code=400, detail="Invalid guest entry")
        name = raw.get("collection", "")
        if not isinstance(name, str) or len(name) > 60:
            raise HTTPException(status_code=400, detail="Invalid collection name")
        output.append((medium, provider_id, item, name.strip()))
    return output

def existing_keys(db: Session, user: User) -> set[tuple[str, str]]:
    return {tuple(row) for row in db.execute(select(
        LibraryEntry.medium, LibraryEntry.provider_id
    ).where(LibraryEntry.user_id == user.id)).all()}}

@router.post("/preview")
def preview(
    payload: dict,
    db: Annotated[Session, Depends(get_db)],
    user: Annotated[User, Depends(current_user)],
) -> dict[str, int]:
    items = read_guest(payload)
    old = existing_keys(db, user)
    new = len({(medium, str(pid)) for medium, pid, _, _ in items}) - old
    return {"new": new, "already_in_account": len(items) - new}

@router.post("/confirm")
def confirm(
    payload: dict,
    db: Annotated[Session, Depends(get_db)],
    login: Annotated[Login, Depends(require_write)],
) -> dict[str, int]:
    user = login[0]
    items = read_guest(payload)

```

```

old = existing_keys(db, user)
current_count = db.scalar(select(func.count()).select_from(LibraryEntry).where(
    LibraryEntry.user_id == user.id
))
incoming = len(({medium, str(pid)} for medium, pid, _, _ in items) - old)
if current_count + incoming > 2000:
    raise HTTPException(status_code=400, detail="Library would exceed 2,000 entries")
collections = {row.name: row for row in db.scalars(select(Collection).where(
    Collection.user_id == user.id
)).all()}
added = 0
for medium, pid, item, name in items:
    key = (medium, str(pid))
    if key in old:
        continue
    collection = None
    if name:
        collection = collections.get(name)
        if collection is None:
            collection = Collection(user_id=user.id, name=name)
            db.add(collection)
            db.flush()
            collections[name] = collection
    db.add(LibraryEntry(
        user_id=user.id, medium=medium, provider_id=str(pid),
        title_snapshot=item.title, status=item.status,
        progress=item.progress, rating=item.rating, favorite=item.favorite,
        notes=item.notes, collection_id=collection.id if collection else None,
        version=1,
    ))
    old.add(key)
    added += 1
db.commit()
return {"inserted": added, "already_in_account": len(items) - added}

```

```

@router.get("/export")
def export(
    db: Annotated[Session, Depends(get_db)],
    user: Annotated[User, Depends(current_user)],
) -> dict:
    names = {row.id: row.name for row in db.scalars(select(Collection).where(
        Collection.user_id == user.id
    )).all()}
    rows = db.scalars(select(LibraryEntry).where(LibraryEntry.user_id == user.id)).all()
    entries = {}
    for row in rows:
        key = f"{row.medium}/{row.provider_id}"
        entries[key] = {
            "title": {"id": int(row.provider_id), "medium": row.medium,
                      "title": row.title_snapshot},
            "status": row.status, "progress": row.progress,
            "rating": int(row.rating or 0), "favorite": row.favorite,
            "notes": row.notes, "collection": names.get(row.collection_id, ""),
            "updatedAt": datetime.now(timezone.utc).isoformat(),
        }
    return {"version": 1, "entries": entries, "recent": [], "compare": []}

```

2. Register the router. In `app/main.py`, import `router` as `imports_router` from `app.api.imports` and add `app.include_router(imports_router)` with the other routers. Save, run the import check, and reopen `/docs`.

3. Check with a throwaway guest export. Export a small guest library file from the frontend. In PowerShell, replace the example path with that file's actual path. Sign in and get `$headers` as in Step 7.1. Send the same JSON to preview and confirm:

```

$guestFile = 'C:\Users\joshj\Downloads\my-guest-library.json'
$json = Get-Content -LiteralPath $guestFile -Raw
Invoke-RestMethod "$api/v1/library-import/preview" -Method Post -ContentType 'application/json' -
-Body $json -WebSession $browser
Invoke-RestMethod "$api/v1/library-import/confirm" -Method Post -ContentType 'application/json' -
-Body $json -WebSession $browser -Headers $headers

```

Preview reports counts without changing `/v1/library`. After confirm, the new rows appear. Run confirm a second time: `inserted` must be 0. The guest file and browser `localStorage` remain unchanged. If you only want to inspect a file, stop after preview; there is no need to run confirm.

☐ Step 7.2 complete - preview is read-only, and repeated import adds zero entries

Phase 8 - Find out what catalog data you may save

Step 8.1 - Make a permission note

Open `kuro-frontend/docs/provider-permissions.md` in your editor. If it does not exist, create it. Paste this template and fill it from the **written** answer for your actual provider plan:

```
# Kuroyume provider permission

Provider and plan:
Date checked:
Link to written terms or reply:

May store title/ID? Yes / No / Unknown
May store summaries? Yes / No / Unknown
May store image URLs or images? Yes / No / Unknown
May store scores, rankings, relations, recommendations? Yes / No / Unknown
May display stored fields to visitors? Yes / No / Unknown
Maximum retention time:
Refresh and bulk-request limits:
Deletion requirements:

Decision: PENDING / PERSISTENT STORAGE ALLOWED / TEMPORARY CACHE ONLY / NO STORAGE
```

Use the provider's terms/dashboard or ask the rights holder these questions. Do not paste an API key, password, or confidential agreement into the file. If any required answer is **Unknown**, leave `Decision: PENDING`. This is a real stop point: account work can continue, but a permanent catalog or sync job cannot be chosen yet.

Check: You can point to a dated written source for each field you plan to save. If you cannot, leave the decision pending and go to Step 10.2, which does not need a provider catalog.

☐ Step 8.1 complete - the decision is based on a written source

Step 8.2 - Make a field list only after permission is clear

If the decision allows persistent storage, open `kuro-frontend/lib/catalog/types.ts` and `lib/catalog/normalize.ts`. In the same permission note, add a table with three columns: **Kuroyume field**, **provider source**, and **allowed to store until**. Start with the fields used by `Detail`: ID, medium, title, image, synopsis, score, year, genres, relations, and characters. Cross out any field the agreement does not allow. Save the note.

Check: You have a short list of actual fields and expiry rules. You have not yet made database tables or provider requests. The exact catalog table and sync code must use the provider's permitted fields and request format; there is no safe universal paste-in implementation before that answer exists.

☐ Step 8.2 complete - permitted fields and expiry are listed

Phase 9 - Catalog storage and sync: wait for the provider answer

This phase is **not a set of coding instructions yet**. The earlier guide told you to create catalog files and a sync runner without knowing the provider's allowed fields, retention window, bulk endpoint, or quota. That would again ask you to invent the missing implementation. The next actions are concrete:

1. Finish Step 8.1 and mark the permission decision.
2. If it says **NO STORAGE** or remains **PENDING**, do not create `catalog` tables or run a sync. The current Next.js catalog can continue to serve public pages under its existing permitted usage. Continue with the account screen in Step 10.2.
3. If it says **TEMPORARY CACHE ONLY**, record the allowed lifetime. A permanent PostgreSQL catalog and bulk sync are outside that decision.
4. If it says **PERSISTENT STORAGE ALLOWED**, obtain the provider's exact endpoint documentation and a sample response you are permitted to use. Then a manual recipe can name real fields, exact requests, table columns, expiry and a safe maximum call count. Do not guess numeric IDs or turn on a schedule.

Check: No provider data is copied into PostgreSQL and no background job is enabled while the decision is pending. This is an intentional boundary, not a hidden homework step.

☐ Phase 9 ready to start only after the written answer and API format are available

Phase 10 - Put the account API on a simple frontend screen

Step 10.1 - Keep public catalog pages working while Phase 9 is pending

Do not change `lib/catalog/service.ts` or the detail page until Phase 9 has a working, permitted backend endpoint. The public pages already use their existing Next.js provider path. You can still finish the account part of the app now. When a catalog backend exists, move **one** detail page first, compare the old and new results, then move search and rankings separately. This step contains no code to paste because the backend endpoint and permission decision do not exist yet.

Check: Open an anime and a manga detail page in the current frontend. They still behave as they did before account work.

☐ Step 10.1 complete - public pages still work

Step 10.2 - Add a first account page by hand

This makes a small page where you can sign in and see the server library. It leaves the existing guest library untouched. You can connect the title-save controls after this page works.

1. Set the local API address. In `kuro-frontend/.env.local`, add this non-secret line. Create the file if needed. If it already exists, add the line without removing other settings.

```
NEXT_PUBLIC_KUROYUME_API_URL=http://localhost:8000
```

In the backend's `.env`, `FRONTEND_ORIGIN` must be `http://localhost:3000` and `COOKIE_SECURE=false` for local HTTP. Do not print the whole `.env` because it contains a database password. Restart both development servers after changing either file.

2. Create `kuro-frontend/lib/api/browser.ts`. Open PowerShell in the frontend, make the folder, then open the file:

```
Set-Location 'C:\Users\joshj\OneDrive\Desktop\Projects\kuroyume\kuro-frontend'
New-Item -ItemType Directory -Force -Path lib\api | Out-Null
notepad .\lib\api\browser.ts
```

Paste and save:

```
const base = process.env.NEXT_PUBLIC_KUROYUME_API_URL ?? "http://localhost:8000";

export async function apiGet<T>(path: string): Promise<T> {
  const response = await fetch(`${base}${path}`, { credentials: "include" });
  if (!response.ok) throw new Error(`API returned ${response.status}`);
  return (await response.json()) as T;
}
```



```

export async function apiPost<T>(path: string, body: unknown): Promise<T> {
  const response = await fetch(`${base}${path}`, {
    method: "POST", credentials: "include",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(body),
  });
  if (!response.ok) throw new Error(`API returned ${response.status}`);
  return (await response.json()) as T;
}

export async function apiWrite<T>(
  path: string, method: "POST" | "PUT" | "DELETE", body?: unknown,
): Promise<T> {
  const token = await apiGet<{ csrf_token: string }>("/v1/auth/csrf");
  const response = await fetch(`${base}${path}`, {
    method, credentials: "include",
    headers: {
      "Content-Type": "application/json",
      "X-CSRF-Token": token.csrf_token,
    },
    body: body === undefined ? undefined : JSON.stringify(body),
  });
  if (!response.ok) throw new Error(`API returned ${response.status}`);
  return (response.status === 204 ? undefined : await response.json()) as T;
}

```

The browser itself supplies the `Origin` header; JavaScript should not try to set it. `credentials: "include"` tells it to send the login cookie.

3. Create `kuro-frontent/app/account/page.tsx`. Make the folder with `New-Item -ItemType Directory -Force app\account`, open `app\account\page.tsx`, and paste:

```

"use client";

import { useCallback, useEffect, useState } from "react";
import { apiGet, apiPost, apiWrite } from "@lib/api/browser";

type User = { id: string; email: string; verified: boolean };
type Entry = {
  id: string; medium: "anime" | "manga"; provider_id: number;
  title: string; status: string; progress: number; rating: number;
};

export default function AccountPage() {
  const [user, setUser] = useState<User | null>(null);
  const [entries, setEntries] = useState<Entry[]>([]);
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [message, setMessage] = useState("Checking your account...");

  const refresh = useCallback(async () => {
    try {
      const me = await apiGet<User>("/v1/me");
      setUser(me);
      setEntries(await apiGet<Entry[]>("/v1/library"));
      setMessage("");
    } catch {
      setUser(null);
      setEntries([]);
      setMessage("Sign in to see your account library.");
    }
  }, []);

  useEffect(() => { void refresh(); }, [refresh]);

  async function signIn() {
    try {
      await apiPost<User>("/v1/auth/login", { email, password });
      setPassword("");
      await refresh();
    } catch {

```

```

    setMessage("Sign in failed. Check your email and password.");
  }
}

async function signOut() {
  await apiWrite("/v1/auth/logout", "POST");
  await refresh();
}

return (
  <main style={{ maxWidth: 700, margin: "3rem auto", padding: "1rem" }}>
    <h1>Your account library</h1>
    <p role="status">{message}</p>
    {!user ? (
      <form onSubmit={(event) => { event.preventDefault(); void signIn(); }}>
        <label>Email <input type="email" value={email}
          onChange={(event) => setEmail(event.target.value)} required /></label>
        <label>Password <input type="password" value={password}
          onChange={(event) => setPassword(event.target.value)} required /></label>
        <button type="submit">Sign in</button>
      </form>
    ) : (
      <>
        <p>Signed in as {user.email}</p>
        <button type="button" onClick={() => void signOut()}>Sign out</button>
        <h2>Saved titles</h2>
        {entries.length === 0 ? <p>No account titles saved yet.</p> : (
          <ul>{entries.map((entry) => <li key={entry.id}>
            {entry.title} - {entry.status} ({entry.progress})
          </li>)}</ul>
        )}
      </>
    )}
  </main>
);
}

```

4. Run and check. Because this project was moved, first check that its local Node.js package links still work:

```
Test-Path .\node_modules\typescript\lib\typescript.js
```

If it prints `False`, run `pnpm install --frozen-lockfile` once from `kuro-frontent` to rebuild those links. Then start the backend in one window and `pnpm dev` from `kuro-frontent` in another. Open <http://localhost:3000/account>. Before login, it shows a sign-in form. After login, it shows the server library entry you created in Step 7.1. Sign out and reload: the account entries disappear from this page, while the guest library at `/library` is unchanged. If the browser shows a CORS error, check the exact `FRONTEND_ORIGIN` and restart the backend.

```
pnpm lint
pnpm build
```

Both checks should finish successfully. If they fail, use the first error's filename and line to find the issue in the code you pasted.

□ Step 10.2 complete - account page works and the guest library is unchanged

Step 10.3 - Make the existing save buttons choose the right library

Step 10.2 proves the account API in a separate screen. This step connects the existing title cards, detail-page tracker, and `/library` page. Work in `kuro-frontent`; keep the backend running. Save each file before running the checks.

1. Create `lib/api/account-context.tsx`. This asks `/v1/me` once when the app loads. It shows a loading state until it knows whether the visitor is a guest or signed in. The safe default outside the provider keeps existing component tests in guest mode.

```
"use client";
```

```

import { createContext, useCallback, useContext, useEffect, useState, type ReactNode } from "react";
import { apiGet } from "@lib/api/browser";

export type AccountEntry = {
  id: string; medium: "anime" | "manga"; provider_id: number;
  title: string; status: string; progress: number; rating: number;
  favorite: boolean; notes: string; collection_id: string | null; version: number;
};
type Mode = "loading" | "guest" | "account" | "error";
type AccountState = { mode: Mode; entries: AccountEntry[]; refresh: () => Promise<void> };
const Context = createContext<AccountState>({
  mode: "guest", entries: [], refresh: async () => {},
});

export function AccountProvider({ children }: { children: ReactNode }) {
  const [mode, setMode] = useState<Mode>("loading");
  const [entries, setEntries] = useState<AccountEntry[]>([]);
  const refresh = useCallback(async () => {
    try {
      await apiGet("/v1/me");
      const all: AccountEntry[] = [];
      for (let offset = 0; offset < 2000; offset += 100) {
        const page = await apiGet<AccountEntry[]>(`/v1/library?limit=100&offset=${offset}`);
        all.push(...page);
        if (page.length < 100) break;
      }
      setEntries(all);
      setMode("account");
    } catch (error) {
      const message = error instanceof Error ? error.message : "";
      setEntries([]);
      setMode(message === "API returned 401" ? "guest" : "error");
    }
  }, []);
  useEffect(() => { void refresh(); }, [refresh]);
  return <Context.Provider value={{ mode, entries, refresh }}>
    {children}
  </Context.Provider>;
}

export function useAccount() { return useContext(Context); }

```

2. Wrap the app once. In `app/layout.tsx`, add this import near the other imports:

```
import { AccountProvider } from "@lib/api/account-context";
```

Inside `<body>`, put `<AccountProvider>` around the existing `<SiteHeader />`, `<main>...</main>`, and `<SiteFooter />`. Do not remove any of those three. The body should have this shape:

```

<body className="flex min-h-full flex-col">
  <AccountProvider>
    <SiteHeader />
    <main id="main-content" className="flex-1">{children}</main>
    <SiteFooter />
  </AccountProvider>
</body>

```

3. Create `components/library/account-controls.tsx`. It provides the signed-in versions of the save button, tracker, and library list. A failed save stays visible. A 409 tells the visitor to reload instead of silently overwriting another device's edit.

```

"use client";

import Link from "next/link";
import { useEffect, useState } from "react";
import type { Title } from "@lib/catalog/types";
import { apiGet, apiPost, apiWrite } from "@lib/api/browser";
import { useAccount, type AccountEntry } from "@lib/api/account-context";
import { useLibrary } from "@lib/library/store";

```

```

function path(title: Title) { return `/v1/library/${title.medium}/${title.id}`; }
function find(entries: AccountEntry[], title: Title) {
  return entries.find((e) => e.medium === title.medium && e.provider_id === title.id);
}
function body(title: Title, entry?: AccountEntry) {
  return {
    title: title.title, status: entry?.status ?? "planned",
    progress: entry?.progress ?? 0, rating: entry?.rating ?? 0,
    favorite: entry?.favorite ?? false, notes: entry?.notes ?? "",
    collection_id: entry?.collection_id ?? null,
    version: entry?.version ?? null,
  };
}
function explain(error: unknown) {
  if (error instanceof Error && error.message.includes("409"))
    return "This entry changed elsewhere. Reload and try again.";
  return "The save failed. Your previous account entry is still there.";
}

export function AccountSaveButton({ title }: { title: Title }) {
  const { entries, refresh } = useAccount();
  const entry = find(entries, title);
  const [busy, setBusy] = useState(false);
  const [notice, setNotice] = useState("");
  async function toggle() {
    setBusy(true); setNotice("");
    try {
      if (entry) await apiWrite(path(title), "DELETE");
      else await apiWrite(path(title), "PUT", body(title));
      await refresh();
    } catch (error) { setNotice(explain(error)); }
    finally { setBusy(false); }
  }
  return <span>
    <button type="button" disabled={busy} onClick={() => void toggle()}
      aria-label={` ${entry ? "Remove" : "Save"} ${title.title} `}
      aria-pressed={!entry}>
        {busy ? "Saving..." : entry ? "Saved" : "Save"}
      </button>
      {notice && <small role="alert">{notice}</small>}
    </span>;
}

export function AccountTracker({ title }: { title: Title }) {
  const { entries, refresh } = useAccount();
  const entry = find(entries, title);
  const [status, setStatus] = useState("planned");
  const [progress, setProgress] = useState(0);
  const [rating, setRating] = useState(0);
  const [favorite, setFavorite] = useState(false);
  const [collectionId, setCollectionId] = useState<string | null>(null);
  const [collections, setCollections] = useState<{ id: string; name: string }[]>([]);
  const [notes, setNotes] = useState("");
  const [notice, setNotice] = useState("");
  const [busy, setBusy] = useState(false);
  useEffect(() => {
    setStatus(entry?.status ?? "planned");
    setProgress(entry?.progress ?? 0);
    setRating(entry?.rating ?? 0);
    setFavorite(entry?.favorite ?? false);
    setCollectionId(entry?.collection_id ?? null);
    setNotes(entry?.notes ?? "");
  }, [entry]);
  useEffect(() => {
    void apiGet<{ id: string; name: string }[]>("/v1/collections")
      .then(setCollections).catch(() => setCollections([]));
  }, []);
  async function save() {
    setBusy(true); setNotice("");
    try {
      await apiWrite(path(title), "PUT", {

```

```

        ...body(title, entry), status, progress, rating, favorite,
        collection_id: collectionId, notes,
    });
    await refresh();
    setNotice("Saved to your account.");
  } catch (error) { setNotice(explain(error)); }
  finally { setBusy(false); }
}
return <section className="tracker">
  <h2>Your account entry</h2>
  <label>Status <select value={status} onChange={(e) => setStatus(e.target.value)}>
    {(title.medium === "anime"
      ? ["planned", "watching", "completed", "on-hold", "dropped"]
      : ["planned", "reading", "completed", "on-hold", "dropped"])
    }.map((value) => <option key={value} value={value}>{value}</option>)}
  </select></label>
  <label>Progress <input type="number" min={0} max={100000}
    value={progress} onChange={(e) => setProgress(Number(e.target.value))} /></label>
  <label>Rating <input type="number" min={0} max={10}
    value={rating} onChange={(e) => setRating(Number(e.target.value))} /></label>
  <label>Favorite <input type="checkbox" checked={favorite}
    onChange={(e) => setFavorite(e.target.checked)} /></label>
  <label>Collection <select value={collectionId ?? ""}
    onChange={(e) => setCollectionId(e.target.value || null)}>
    <option value="">None</option>
    {collections.map((c) => <option key={c.id} value={c.id}>{c.name}</option>)}
  </select></label>
  <label>Private notes <textarea maxLength={4000} value={notes}
    onChange={(e) => setNotes(e.target.value)} /></label>
  <button type="button" disabled={busy} onClick={() => void save()}>
    {busy ? "Saving..." : "Save account entry"}
  </button>
  <Link href="/library">Open account library</Link>
  <p role="status">{notice}</p>
</section>;
}

export function AccountLibraryView() {
  const { entries, refresh } = useAccount();
  const { state: guest } = useLibrary();
  const [preview, setPreview] = useState<{ new: number; already_in_account: number } | null>(null);
  const [notice, setNotice] = useState("");
  async function showPreview() {
    try {
      setPreview(await apiPost<{ new: number; already_in_account: number }>({
        "/v1/library-import/preview", guest,
      }));
      setNotice("");
    } catch { setNotice("The guest library could not be previewed."); }
  }
  async function confirmImport() {
    try {
      await apiWrite("/v1/library-import/confirm", "POST", guest);
      await refresh();
      setPreview(null);
      setNotice("Import complete. Your guest library is still on this device.");
    } catch { setNotice("Import failed. Your guest library is unchanged."); }
  }
  return <main className="page-width platform-page">
    <h1>Your account library</h1>
    {entries.length ? <ul>{entries.map((e) => <li key={e.id}>
      <Link href={`/${e.medium}/${e.provider_id}`}>{e.title}</Link> - {e.status}
    </li>)}</ul> : <p>No account entries yet.</p>}
    {Object.keys(guest.entries).length > 0 && <section>
      <h2>Guest entries on this device</h2>
      <button type="button" onClick={() => void showPreview()}>Preview import</button>
      {preview && <div>
        <p>{preview.new} new; {preview.already_in_account} already in account.</p>
        <button type="button" onClick={() => void confirmImport()}>Import</button>
        <button type="button" onClick={() => setPreview(null)}>Cancel</button>
      </div>}
    }
  </main>
}

```

```

    </div>}
  </section>}
  <p role="status">{notice}</p>
</main>;
}

```

Before saving this file, confirm the title link path in `app/`: if Kuroyume's actual title URLs differ from `/{$e.medium}/{$e.provider_id}`, use the same pattern as its existing cards. This is a **path lookup**, not a new feature design.

4. Switch the existing controls. Open `components/library/controls.tsx`. Change its existing `export function SaveButton` to function `GuestSaveButton` and its existing `export function Tracker` to function `GuestTracker`. Leave the bodies of those two functions alone. Add these imports at the top:

```

import { useAccount } from "@lib/api/account-context";
import { AccountSaveButton, AccountTracker } from "@components/library/account-controls";

```

Add these two exported wrappers at the bottom:

```

export function SaveButton({ title }: { title: Title }) {
  const { mode } = useAccount();
  if (mode === "loading") return <span>Checking account...</span>;
  if (mode === "error") return <span role="alert">Account service unavailable</span>;
  return mode === "account"
    ? <AccountSaveButton title={title} />
    : <GuestSaveButton title={title} />;
}

export function Tracker({ title }: { title: Title }) {
  const { mode } = useAccount();
  if (mode === "loading") return <p>Checking account...</p>;
  if (mode === "error") return <p role="alert">Account service unavailable</p>;
  return mode === "account"
    ? <AccountTracker title={title} />
    : <GuestTracker title={title} />;
}

```

5. Switch the library page. Open `components/library/views.tsx`. Change the existing `export function LibraryView` to function `GuestLibraryView`; leave its body alone. Add these imports:

```

import { useAccount } from "@lib/api/account-context";
import { AccountLibraryView } from "@components/library/account-controls";

```

At the bottom of the file, add:

```

export function LibraryView() {
  const { mode } = useAccount();
  if (mode === "loading") return <p>Checking account...</p>;
  if (mode === "error") return <p role="alert">Account service unavailable</p>;
  return mode === "account" ? <AccountLibraryView /> : <GuestLibraryView />;
}

```

6. Refresh account mode after sign-in/out. Open `app/account/page.tsx` from Step 10.2. Import `useAccount` from `@lib/api/account-context`, then inside `AccountPage` add `const { refresh: refreshAccount } = useAccount()`; In its `signIn` function, immediately after `await refresh()`, add `await refreshAccount()`; Do the same in `signOut`. Save.

7. Check in browser and terminal. Restart `pnpm dev`, sign in at `/account`, open a title page, and save it. `/library` should now show the account entry. Sign out: `/library` should show the old guest entries instead. Sign in again: the account entry returns. Put the backend offline temporarily: controls should say the account service is unavailable rather than writing to guest `localStorage`. Try an old version with the API from Step 7.1; it must return 409. Then run:

```

pnpm test
pnpm lint
pnpm build

```

If a command fails, use its first reported file and line. Existing `SaveButton` tests should remain in guest mode when rendered without `AccountProvider`. The guest library stays unchanged throughout these account actions.

☐ Step 10.3 complete - detail-page saves and /library choose the right mode

Phase 11 - Check what you built before deploying

Step 11.1 - Run the checks that exist now

Do these from the named folders. `compileall` checks Python syntax; the `import` command checks that the API's modules can load. `alembic current` reports the database migration version. `pnpm check` runs the frontend's existing tests, lint, build and secret scan.

```
Set-Location 'C:\Users\joshj\OneDrive\Desktop\Projects\kuroyume\kuro-backend'
.\env\Scripts\python.exe -m compileall -q app
.\env\Scripts\python.exe -c "from app.main import app; print('imports ok')"
.\env\Scripts\python.exe -m alembic current
Set-Location '..\kuro-frontend'
pnpm check
```

Then repeat the manual checks from Steps 6.1, 6.2, 6.3, 7.1 and 7.2 using **throwaway local accounts**. In particular, verify that a second account cannot read the first account's library and an old `version` returns 409. If you create automated backend tests later, point them at the **test** database and refuse to run them when it is not `kuroyume_test`; never use `kuroyume_dev` as a test target.

Check: Every command succeeds and the manual security checks behave exactly as stated. Record the first failure, fix it, and rerun that step before continuing. These checks are a local milestone; they do not replace a full automated backend test suite for production.

☐ Step 11.1 complete - local commands and manual account checks pass

Step 11.2 - Write a deployment checklist you can actually use

Open `kuro-backend/DEPLOYMENT-CHECKLIST.md` and create this short checklist. Fill the blanks with real choices only when you are ready to deploy:

```
# Kuroyume deployment checklist

- Frontend address:
- Backend address and HTTPS certificate:
- PostgreSQL host and backup location:
- Tested command to restore a backup:
- Secret storage location:
- Mail sender for verification/reset:
- Login rate limit and monitoring:
- Migration command and database it will change:
- Health URLs to check after deployment:
- Provider permission note and quota alert (if catalog work is enabled):
- How to disable any sync job:
- Date a staging restore and migration were tested:
```

Do not mark this complete with blank lines. The local `.local-mail/` file from Step 6.2 is **not** a production mail sender. The account page from Step 10.2 does **not** mean the title-page controls are integrated. Schedule a production change only after the missing pieces and automated tests are complete.

☐ Step 11.2 complete - every deployment line has a real, tested answer

When you get stuck

You see	First thing to check
Error loading ASGI app	Are you inside kuro-backend? Does <code>app/main.py</code> define <code>app</code> ? Do package folders have <code>__init__.py</code> ?
connection refused	Is the API or PostgreSQL service running on that host and port?
password authentication failed	Does <code>.env</code> have the right PostgreSQL role and password?
relation private.users does not exist	Is the API using the same database you migrated? What does <code>alembic current</code> show?
FastAPI 422	Read the response's <code>detail</code> ; a request field did not match its schema.
FastAPI 401	Check the cookie, its expiry, and whether the session was revoked.
FastAPI 403 on a write	Check the CSRF header, browser Origin and row ownership.
Browser CORS error	Check the exact frontend origin, cookie host and <code>credentials: 'include'</code> .
Provider 429	Stop requests and check its retry delay and your actual plan dashboard.
Empty Alembic revision	Check model imports and <code>target_metadata</code> in <code>alembic/env.py</code> .

When asking for help, share the **step number**, exact command or URL, error text with secrets removed, and what its **Check** section showed.

Finish line

Local account milestone you can build from this guide

- [] The guest library still works without an account.
- [] `/health` and `/health/db` respond on the local backend.
- [] Signup, login, logout, verification and password reset work with local test accounts.
- [] A second account cannot read or change the first account's library.
- [] Progress, ratings, notes and collections survive a reload.
- [] Guest import requires preview and explicit confirmation.
- [] The frontend shows the guest library when signed out and account entries when signed in.
- [] Python import/syntax checks and `pnpm check` pass.

Separate work before production

- [] Automated backend tests use `kuroyume_test`, never `kuroyume_dev`.
- [] Real email delivery replaces the local test mailbox.
- [] Login and recovery endpoints have rate limits and monitoring.
- [] HTTPS, backups, tested restore, and migration procedure are ready.
- [] Provider permission is documented before catalog storage or sync.
- [] The public catalog moves to FastAPI only after a permitted backend route exists.

Reference shelf

Open these only when you reach the matching step: [FastAPI first steps](#), [FastAPI dependencies](#), [FastAPI CORS](#), [FastAPI password hashing](#), [PostgreSQL tutorial](#), [SQLAlchemy sessions](#), [Alembic tutorial](#), and [GitHub Actions syntax](#), and [OWASP CSRF guidance](#).